01-01-python-setup

January 7, 2026

```
<div style='float:left;'><h1>CPSC 4300/6300: Applied Data Science</h1></div>
<img style="float: right; padding-right: 10px; width: 65px" src="https://raw.githubusercontent
```

1 Week 1 | Lab 1: Environment Setup

Clemson University Instructor: Tim Ransom

1.1 Learning Objective

- Explain the purpose of virtual environments in Python.
- Install Python modules using conda on your local machine.

[ ]:
```
""" RUN THIS CELL TO GET THE RIGHT FORMATTING """
import requests
from IPython.core.display import HTML
css_file = 'https://raw.githubusercontent.com/bsethwalker/clemson-cs4300/main/
 ↪css/cpsc6300.css'
styles = requests.get(css_file).text
HTML(styles)
```

1.2 1 - Introduction to Python and Jupyter Notebook Setup

- To complete this course, you will be using Python and several popular third-party libraries for scientific computing.
- We strongly recommend setting up a local development environment so you can access the notebook files offline
- This guide walks you through setting up Python, Anaconda, and Jupyter Notebook, which will be your main tool for interactive coding.

Key Points:

1. **Python Version:** Use `Python 3`, **not** `Python 2`.
2. **Jupyter Notebook:** Works best in `Microsoft Edge`, `Google Chrome`, `Firefox`.

1.3 2 - Installing Anaconda

The Anaconda distribution simplifies setting up Python and includes all the necessary libraries for this course. We recommend installing it unless you have a specific reason not to.

1.3.1 2.1 Mac/Linux Users

- **Download Anaconda**:
  - Visit the Anaconda download page and download the installer for your OS.
- **Install Anaconda**:
  - Follow the instructions on the download page to run the installer.
- **Test Jupyter Notebook**:
  - Open a Terminal window and type:
    `jupyter notebook`
  - Alternatively, use the Anaconda Launcher to start Jupyter Notebook.
  - A new browser window should open.
  - Click **New Notebook** and give it a unique name.
- **Working in a Specific Folder**:
  - Open a terminal and navigate to the desired folder using:
    `cd Documents/cpsc4300`
  - Start Jupyter Notebook in that folder:
    `jupyter notebook`

1.3.2 2.2 Windows Users

- **Download Anaconda**:
  - Visit Anaconda download page.
  - Download the correct version for Windows.
- **Install Anaconda**:
  - Follow the instructions provided during the installation. Typically, Anaconda will be installed in `C:\Anaconda`.
- **Test Jupyter Notebook**:
  - Start the Anaconda launcher from the installation directory or the Start menu.
  - Launch Jupyter Notebook, and a new browser window should open.
  - Create a new notebook with a unique name and search for the file in Explorer to find the default folder location.
    - **Trick**: give this notebook a unique name, like `my-little-rose`. Use Explorer (usually start menu on windows desktops) to search for this name. In this way, you will know which folder your notebook opens in by default.
- **Install Git-Bash (Recommended)**:
  - Git-Bash provides a convenient terminal for Windows users.

1.3.3 2.3 Common for All Users

- **Adding Anaconda to PATH**:
  - If Anaconda was not added to your path, use the full path to Python, such as `/anaconda/bin/python`.
- **Update Anaconda**:
  - If you have installed Anaconda previously, update to the latest version:

```
conda update conda
conda update anaconda
```

1.4 3 - Hello, Jupyter

- The Jupyter Notebook is a web application that allows you to create interactive documents that contain live code, equations, visualizations and explanatory text.
- When Jupyter app loads, you see a dashboard displaying files in the Jupyter home directory (you can reset this)
- Each notebook consists of blocks of cells. Each cell can display rich text elements (Markdown) or code. Code is executed by a "computational engine" called the **kernel** . The output of the code is displayed directly below.
- Each cell can be executed independently, but once a block of code is executed, it lives in the memory of the kernel.
- You'll be using Jupyter notebooks within Coursera Lab environment in order to complete labs and homework. Once you've set up Python, please download this page, and open it with Jupyter by typing

`jupyter notebook <name_of_downloaded_file>`

As mentioned earlier in the Mac section, you can also open the notebook in any folder by changing directory using `cd` command to the folder in the terminal, and typing

`jupyter notebook .`

in that folder.

The anaconda install also probably dropped a launcher on your desktop. You can use the launcher, and select "jupyter notebook" from there. In this case you will need to find out which folder you are running in.

Notice that you can use the user interface to create new folders and text files, and even open new terminals, all of which might come useful to you. To create a new notebook, you can use "Python 3" under notebooks. You may not have the other choices available (I have julia for example, which is another language that uses the same notebook interface).

For the rest of this setup test, use your local copy of this page, running on jupyter.

Notebooks are composed of many "cells", which can contain text (like this one), or code (like the one below). Double click on the cell below, and evaluate it by clicking the "play" button above, for by hitting shift + enter

You must be careful to make sure you are running the Anaconda version of python, since those operating systems come preinstalled with their own versions of python.

This is how you can see the version in the jupyter interface

[ ]:
```
import sys
print(sys.version)
```

You could also open a terminal and just type

`python`

or

`ipython`

there. When the program starts up, you should see "Anaconda" printed out, similar to the above. If this is the case, your install went well, and you can quit the python "interpreter" by typing Ctrl-D.

If you've successfully completed the above install, skip the below troubleshooting section. All of the statements there should run.

1.5 4 - Troubleshooting

PROBLEM You are using a Mac or Linux computer. When you start python at the terminal or do `sys.version` in the notebook, you don't see a line like `3.5.3 |Anaconda custom (x86_64)| (default, Mar  6 2017, 12:15:08)`.

Reason You are most likely running a different version of Python, and need to modify your Path (the list of directories your computer looks through to find programs).

Solution Find a file like `.bash_profile`, `.bashrc`, or `.profile`. Open the file in a text editor, and add a line at this line at the end:

`export PATH`"$HOME/anaconda/bin:$PATH"=.

Close the file, open a new terminal window, type `source ~/.profile` (or whatever file you just edited). Type

`which python`

– you should see a path that points to the anaconda directory. If so, running `python` should load the proper version.

If this doesn't work (typing `which python` doesn't point to anaconda), you might be using a different shell.

Type `echo $SHELL`.

If this isn't `bash`, you need to edit a different startup file (for example, if `echo $SHELL` gives `$csh`, you need to edit your `.cshrc` file. The syntax for this file is slightly different:

`set PATH = ($HOME/anaconda/bin $PATH)`

PROBLEM You are running the right version of python (see above item), but are unable to import numpy.

Reason You are probably loading a different copy of numpy that is incompatible with Anaconda.

Solution See the above item to find your `.bash_profile`, `.profile`, or `.bashrc` file. Open it, and add the line `unset PYTHONPATH` at the end. Close the file, open a new terminal window, type `source ~/.profile` (or whatever file you just edited), and try again.

PROBLEM Under Windows, you receive an error message similar to the following: " 'pip' is not recognized as an internal or external command, operable program or batch file."

Reason The correct Anaconda paths might not be present in your PATH variable, or Anaconda might not have installed correctly.

Solution Ensure the Anaconda directories to your path environment variable (" " and " "). See this page for details.

If this does not correct the problem, re-install Anaconda.

IF YOU ARE STILL HAVING ISSUES ON THE INSTALL, REACH OUT TO THE COURSE STAFF FOR HELP!

1.6 5 - Environments and Python Libraries

There are two main installing packages for Python, `conda` and `pip`. Pip is the Python Packaging Authority's recommended tool for installing packages from the **Python Package Index (PyPI)**. `Conda` is a cross platform package and environment manager that installs and manages conda packages from the **Anaconda repository** and **Anaconda Cloud**. Conda does not assume any specific configuration in your computer and will install the Python interpreter along with the other Python packages, whereas `pip` assumes that you have installed the Python interpreter in your computer. Given the fact that most operating systems do include Python this is not a problem.

If I could summarize their differences into a sentence it would be that conda has the ability to create **isolated environments** that can contain different versions of Python and/or the packages installed in them. This can be extremely useful when working with data science tools as different tools may contain conflicting requirements which could prevent them all being installed into a single environment. You can have environments with pip but would have to install a tool such as virtualenv or venv. You may use either, we recommend `conda` because in our experience it leads to fewer incompatibilities between packages and thus fewer broken environments.

Conclusion: Use Both. Most often in our data science environments we want to combining pip with conda when one or more packages are only available to install via pip. Although thousands of packages are available in the Anaconda repository, including the most popular data science, machine learning, and AI frameworks but a lot more are available on PyPI. Even if you have your environment installed via `conda` you can use `pip` to install individual packages

(source: anaconda site)

1.6.1 What are environments and do I need them?

Environments in Python are like sandboxes that have different versions of Python and/or packages installed in them. You can create, export, list, remove, and update environments. Switching or moving between environments is called activating the environment. When you are done with an environments you may deactivate it.

For this class we want to have a bit more control on the packages that will be installed with the enviromnent so we will create an environment specifically for this course.

1. Creating an environment

   You can create a new environment by running the following command in the terminal.

`conda create -n cpsc6300 python=3.8`

2. Activate the new environment:

   `source activate cpsc6300`

   You should see the name of the environment at the start of your command prompth in parenthesis.

3. Verify that the new environment was installed correctly:

   `conda list`

   This will give you a list of the packages installed in this environment.

4. References

   Manage conda environments

1.6.2 Starting the Jupyter Notebook

Once all is installed, and your environment is active, go in the Terminal and type

`jupyter notebook`

to start the jupyter notebook server. This will spawn a process that will be running in the Terminal window until you are done working with the notebook. In that case press `control-C` to stop it.

Starting the notebook will bring up a browser window with your file structure.

For more on using the Notebook see: https://jupyter-notebook.readthedocs.io/en/latest/

1.6.3 Installing Modules

We will use specific Python Modules in this course. You can find installation instructions for most modules online, but installing a new module will typically follow this pattern:

`conda install <module_name>`

Before installing the module, make sure the virtual environment in which you want to install it is active. For example, to install Numpy, you would do the following:

```
# Activate environment
conda activate cpsc4300

# Install numpy
conda install numpy
```

1.6.4 Testing latest libraries

Run the cell below to print the version you have install for some key libraries we will use in this course. For reference, I included the version installed on this `Coursera Lab Environment`. Packages are frequently updated, so you don't need to have the exact versions installed like here. However, the versions you're using should be close to, or newer than, currently being used in this lab.

[3]:
```
#IPython is what you are using now to run the notebook
import IPython
print("IPython version:      %6.6s" % IPython.__version__)

# Numpy is a library for working with Arrays
import numpy as np
print("Numpy version:        %6.6s" % np.__version__)

# SciPy implements many different numerical algorithms
import scipy as sp
print("SciPy version:        %6.6s" % sp.__version__)

# Pandas makes working with data tables easier
import pandas as pd
print("Pandas version:       %6.6s" % pd.__version__)

# Module for plotting
import matplotlib
print("Matplotlib version:    %6.6s" % matplotlib.__version__)

# SciKit Learn implements several Machine Learning algorithms
import sklearn
print("Scikit-Learn version: %6.6s" % sklearn.__version__)

# Requests is a library for getting data from the Web
import requests
print("requests version:     %6.6s " % requests.__version__)

import seaborn
print("Seaborn version: %6.6s" % seaborn.__version__)
```

```
IPython version:      8.21.0
Numpy version:        1.25.2
SciPy version:         1.9.3
Pandas version:        1.5.1
Matplotlib version:     3.6.2
Scikit-Learn version:  1.5.2
requests version:     2.32.3
Seaborn version: 0.12.1
```

1.7 6 - Kicking the tires

Lets try some things, starting from very simple, to more complex.

1.7.1 6.1 - Hello World

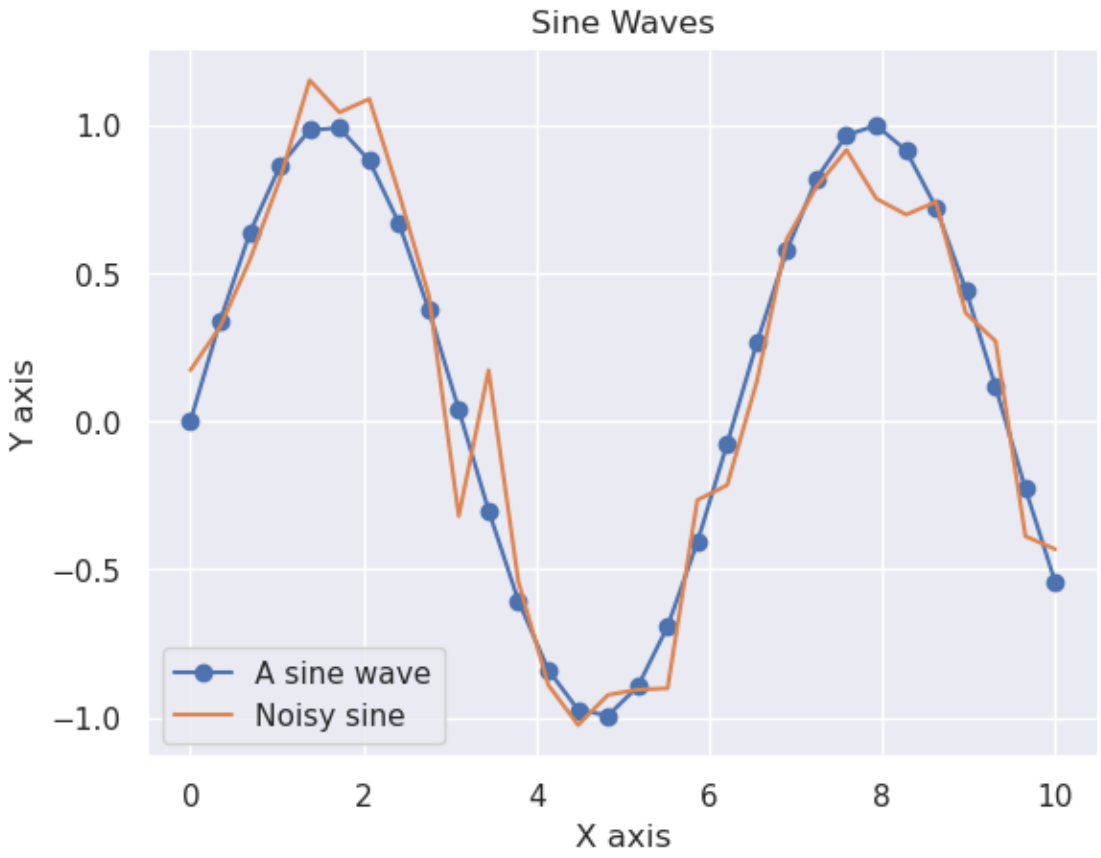
The following is the incantation we like to put at the beginning of every notebook. It loads most of the stuff we will regularly use.

[5]:
```
# The %... is an iPython thing, and is not part of the Python language.
# In this case we're just telling the plotting library to draw things on
# the notebook, instead of on a separate window.
%matplotlib inline
#this line above prepares the jupyter notebook for working with matplotlib

# See all the "as ..." contructs? They're just aliasing the package names.
# That way we can call methods like plt.plot() instead of matplotlib.pyplot.
 ↪plot().
# notice we use short aliases here, and these are conventional in the python␣
 ↪community

import numpy as np              # imports a fast numerical programming library
import scipy as sp              # imports stats functions, amongst other things
import matplotlib as mpl        # this actually imports matplotlib
import matplotlib.cm as cm      # allows us easy access to colormaps
import matplotlib.pyplot as plt # sets up plotting under plt
import pandas as pd             # lets us handle data as dataframes

import seaborn as sns # gives us more plotting options
sns.set()             # sets up styles
```

1.7.2 6.2 - Hello matplotlib

The notebook integrates nicely with Matplotlib, the primary plotting package for python. This should embed a figure of a sine wave:

[6]:
```
x = np.linspace(0, 10, 30)  # array of 30 points from 0 to 10
y = np.sin(x)
z = y + np.random.normal(size=30) * .2

plt.plot(x, y, 'o-', label='A sine wave')
plt.plot(x, z, '-', label='Noisy sine')
plt.legend(loc = 'best')
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.title("Sine Waves");
```



1.7.3 6.3 - Hello Numpy

The Numpy array processing library is the basis of nearly all numerical computing in Python. Here's a 30 second crash course. For more details, consult the Numpy Documentation.

[ ]:
```
print("Make a 3 row x 4 column array of random numbers")
x = np.random.random((3, 4))
print(x,"\n")


print("Add 1 to every element")
x = x + 1
print(x,"\n")

print("Get the element at row 1, column 2")
print(x[1, 2])

# The colon syntax is called "slicing" the array.
print("Get the first row")
print(x[0, :])
```

```
print("\nLast 2 items in the first row")
print(x[0, -2:])

print("\nGet every 2nd item in the first row")
print(x[0, ::2])
```

Print the maximum, minimum, and mean of the array. This does **not** require writing a loop. In the code cell below, type `x.m<TAB>`, to find built-in operations for common array statistics like this

[ ]:
```
print("Max is  ", x.max())
print("Min is  ", x.min())
print("Mean is ", x.mean())
```

Call the `x.max` function again, but use the `axis` keyword to print the maximum of each row in x.

[ ]:
```
print(x.max(axis=1))
```

Here's a way to quickly simulate 500 coin "fair" coin tosses (where the probabily of getting Heads is 50%, or 0.5)

[ ]:
```
x = np.random.binomial(500, .5)
print("number of heads:", x)
```

Repeat this simulation 500 times, and use the plt.hist() function to plot a histogram of the number of Heads (1s) in each simulation

[2]:
```
# 3 ways to run the simulations

# loop
heads = []
for i in range(500):
    heads.append(np.random.binomial(500, .5))

# "list comprehension"
heads = [np.random.binomial(500, .5) for i in range(500)]

# pure numpy, preferred
heads = np.random.binomial(500, .5, size=500)

plt.hist(heads, bins=10);
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 6
      4 heads = []
      5 for i in range(500):
----> 6     heads.append(np.random.binomial(500, .5))
```

```
      8 # "list comprehension"
      9 heads = [np.random.binomial(500, .5) for i in range(500)]

NameError: name 'np' is not defined
```

Finally! Here is an example of an autograded code cell. You should see something along the lines of `# your code here` or `raise NotYetImplementedError` in there. That is your cue where to edit the notebook files for your grade. Go ahead and put an integer into a variable called `answer` in the cell block and hit submit to see this in action.

Underneath, you will see an empty cell block. This is where the instructor code to check your answers are.

[ ]:
```
# your code here
raise NotImplementedError
```

[ ]:
```

```

2 END